

SEND5232: Software Architecture and Design Project

Bereket Kiros – ugr/189518/16 – Section 2

Phase 5 — Architectural Redesign

Lihq'et Bookstore System

1. Introduction

The architectural redesign of the Lihq'et Bookstore System focuses on improving the initial system structure by applying well-established architectural and design patterns. While the initial design provided a functional CRUD-based system, the redesign enhances the system by enforcing strict layering, improving modularity, and aligning the implementation more closely with Architecturally Significant Requirements (ASRs).

The redesigned architecture ensures better separation of concerns, improved scalability, and easier maintenance, making the system more robust and extensible for future enhancements.

2. Refined Architectural Structure

The system continues to follow a **3-tier layered architecture**, but the internal organization of the backend is refined using **MVC (Model-View-Controller)** combined with a **Service Layer pattern**.

◆ Layers:

1. Presentation Layer (Frontend)

- Built using HTML, CSS, and JavaScript
 - Handles user interaction and UI rendering
 - Sends asynchronous HTTP requests using Fetch API
 - Receives JSON responses and updates UI dynamically
-

2. Application Layer (Backend - Node.js + Express)

This layer is restructured into three distinct components:

Controller Layer

- Handles incoming HTTP requests
- Defines RESTful endpoints (/books)
- Delegates processing to the Service Layer
- Returns responses to the frontend

Service Layer (New Emphasis)

- Centralized business logic
- Performs input validation and data processing
- Coordinates between Controller and Model
- Ensures consistency across operations

Model Layer

- Handles database access
 - Executes SQL queries
 - Interacts with MySQL database
-

3. Data Layer

- MySQL Database
 - Stores persistent data (books table)
 - Ensures data integrity and structured storage
-

3. Application of Design Patterns

The redesigned architecture explicitly incorporates the following patterns:

MVC Pattern

Separates system responsibilities into:

- Controller → request handling
- Service (extended logic layer) → business logic
- Model → data access

This separation reduces code complexity and improves maintainability.

Service Layer Pattern

The Service Layer acts as an intermediary between Controller and Model:

- Prevents duplication of logic
 - Ensures consistent validation
 - Supports reusability of core operations
-

Layered Architecture Pattern

The system is divided into:

- Presentation Layer
- Application Layer
- Data Layer

This improves scalability and allows independent evolution of each layer.

In addition to **architectural patterns**, the system applies several design patterns to improve code structure, reusability, and maintainability.

1. Controller Pattern

The Controller acts as an intermediary between the frontend and backend logic. It receives HTTP requests, processes input, and delegates tasks to the Service Layer. This ensures that request handling is separated from business logic.

2. Service Layer Pattern (Behavioral / Structural Hybrid)

The Service Layer encapsulates business logic and validation rules. It ensures that Controllers remain lightweight and prevents duplication of logic across different endpoints. This improves maintainability and consistency.

3. Data Access Pattern (Model Pattern)

The Model component follows a Data Access pattern by isolating all database interactions. It executes SQL queries and abstracts database operations from the rest of the system, ensuring data handling is centralized and controlled.

4. Facade Pattern (Implicit)

The Controller + Service Layer together act as a Facade for the frontend. The frontend interacts with a simplified API interface without needing to understand internal system complexity such as validation, business rules, or database operations.

5. Singleton Pattern (Runtime Behavior)

The database connection (MySQL connection instance) is typically managed as a single shared instance across the application. This ensures efficient resource usage and consistent database access.

4. Improved Data Flow and Interaction

The redesigned system enforces a strict interaction flow:

1. User interacts with frontend UI
2. Frontend sends HTTP request to Controller
3. Controller forwards request to Service Layer
4. Service Layer validates and processes logic
5. Service calls Model for database operations
6. Model executes SQL queries on MySQL
7. Results flow back through Service → Controller → Frontend

This structured pipeline ensures:

- No direct access between frontend and database
 - Controlled data processing
 - Clear responsibility boundaries
-

5. Alignment with ASRs

The redesigned architecture directly addresses the system's ASRs:

- **Performance:** Efficient request handling through layered processing
 - **Security:** Centralized validation in Service Layer reduces risks
 - **Maintainability:** Modular design simplifies debugging and updates
 - **Scalability:** New features can be added without affecting existing components
 - **Usability:** Reliable backend ensures smooth frontend interaction
-

6. Architectural Improvements Over Initial Design

Compared to the initial design, the redesigned architecture introduces:

- Clear separation between Controller, Service, and Model
 - Centralized business logic for better consistency
 - Reduced coupling between components
 - Improved code reusability
 - Better preparation for future system expansion
-

Updated Component Diagram (Redesigned Architecture)

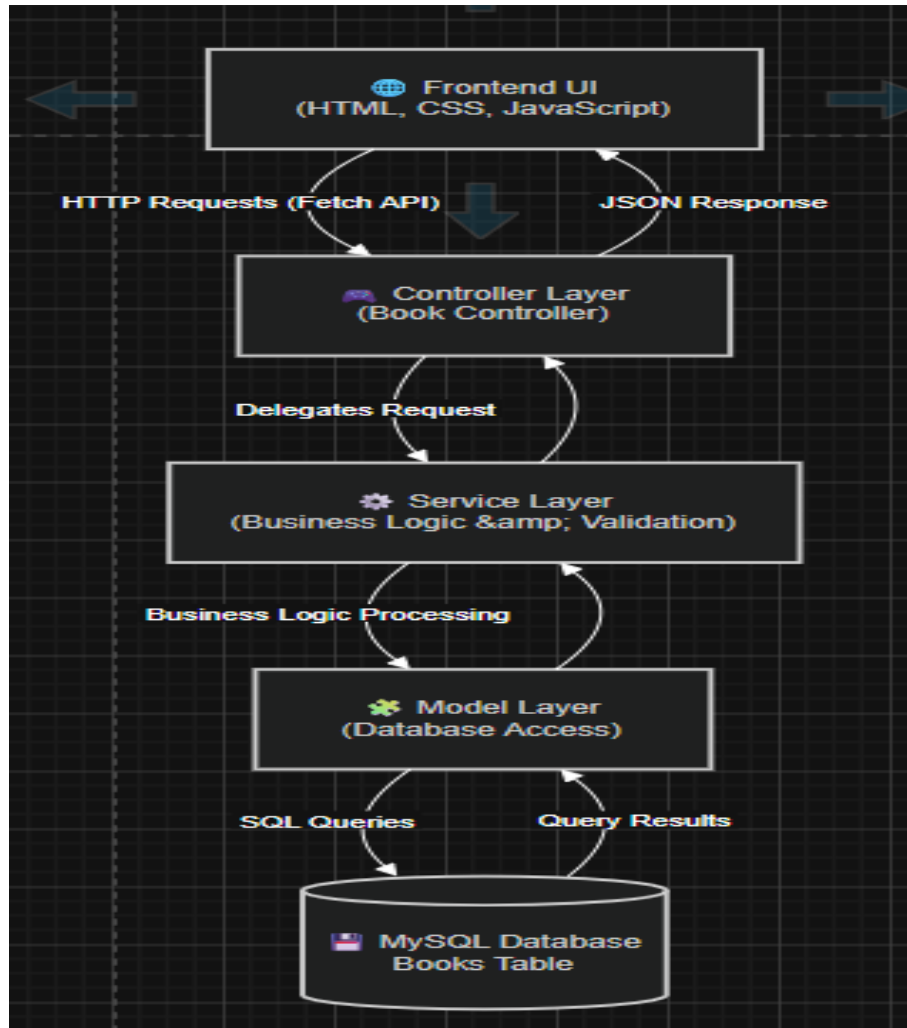


Figure 1: Updated Component Diagram (Redesigned Architecture)

The updated component diagram illustrates the refined internal structure of the Lihq'et Bookstore System after applying MVC and Service Layer patterns. The system is divided into Presentation, Application, and Data layers. The frontend interacts with the Controller through HTTP requests, which delegates processing to the Service Layer. The Service Layer handles business logic and communicates with the Model for database operations. The Model interacts with the MySQL database. This structured flow ensures clear separation of concerns, improved maintainability, and controlled data interaction.